

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования

ТОМСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
СИСТЕМ УПРАВЛЕНИЯ И РАДИОЭЛЕКТРОНИКИ (ТУСУР)

**Кафедра автоматизированных систем управления
(АСУ)**

РЕАЛИЗАЦИЯ АРИФМЕТИЧЕСКИХ ОПЕРАЦИЙ НАД ВРЕМЕНЕМ
СУТОК С ИСПОЛЬЗОВАНИЕМ КЛАССОВ C++

Курсовая работа по дисциплине «Объектно-ориентированное
программирование» (Вариант 13)

Выполнила:
Студентка гр. 3-433П8-5

Медведева Юлия Евгеньевна

«12» апреля 2026г.
(дата)

Проверил:

_____ (должность, ученая степень, звание)	
_____ (подпись)	_____ И. О. Фамилия
« ____ »	20 ____ г. (дата)

Задание

- 1) Реализовать программу, в которой время задается в формате час, минута, секунда
 - a. Вычитание из времени указанного пользователем количества секунд;
 - b. Подсчёт числа секунд между двумя моментами времени, лежащими в пределах одних суток.
- 2) Подготовить ответ на теоретический вопрос «Параметры членов-функций. Способы передачи значений в тело и из тела функции».

СОДЕРЖАНИЕ

Задание.....	2
Введение	4
1 БЛОК-СХЕМА РЕШЕНИЯ ЗАДАЧИ	5
2 ОПИСАНИЕ ПРОГРАММЫ	8
2.1 ОПИСАНИЕ СТРУКТУРЫ ВХОДНЫХ И ВЫХОДНЫХ ДАННЫХ.....	8
2.1.1 Входные данные.....	8
2.1.2 Выходные данные	8
2.2 ФУНКЦИОНАЛЬНАЯ СТРУКТУРА ПРОГРАММЫ	9
2.3 ТЕСТИРОВАНИЕ ПРОГРАММЫ.....	12
2.4 РУКОВОДСТВО ПОЛЬЗОВАТЕЛЯ	22
3 ПАРАМЕТРЫ ЧЛЕНОВ-ФУНКЦИЙ. СПОСОБЫ ПЕРЕДАЧИ ЗНАЧЕНИЙ В ТЕЛО И ИЗ ТЕЛА ФУНКЦИИ	26
3.1 ПАРАМЕТРЫ ЧЛЕНОВ-ФУНКЦИЙ В C++	26
3.2 СПОСОБЫ ПЕРЕДАЧИ ЗНАЧЕНИЙ В ТЕЛО ФУНКЦИИ.....	28
3.3 СПОСОБЫ ПЕРЕДАЧИ ЗНАЧЕНИЙ ИЗ ТЕЛА ФУНКЦИИ	31
Заключение.....	34
Список литературы.....	35

Введение

Целью работы является разработка программы на языке C++ для работы со временем, представленным в формате часы, минуты, секунды.

Основные задачи:

- разработать класс Time для хранения времени и выполнения операций над ним;
- реализовать вычитание заданного количества секунд из момента времени;
- реализовать подсчёт количества секунд между двумя моментами времени в пределах одних суток;
- организовать консольное меню для выбора операции пользователем;
- разработать класс Group для работы с набором объектов Time.

Программа реализована с применением объектно-ориентированного программирования. В теоретической части рассмотрены параметры членов-функций и способы передачи значений в тело и из тела функции в языке C++.

1 БЛОК-СХЕМА РЕШЕНИЯ ЗАДАЧИ

На рисунке 1 изображена общая блок-схема программы. Программа реализует цикличное меню: после запуска пользователю предлагается выбрать одно из трёх действий. Выбранная операция выполняется, после чего меню отображается повторно. Цикл завершается при вводе значения 0. Операции реализованы в виде отдельных подпрограмм, блок-схемы которых представлены на рисунках 2 и 3.

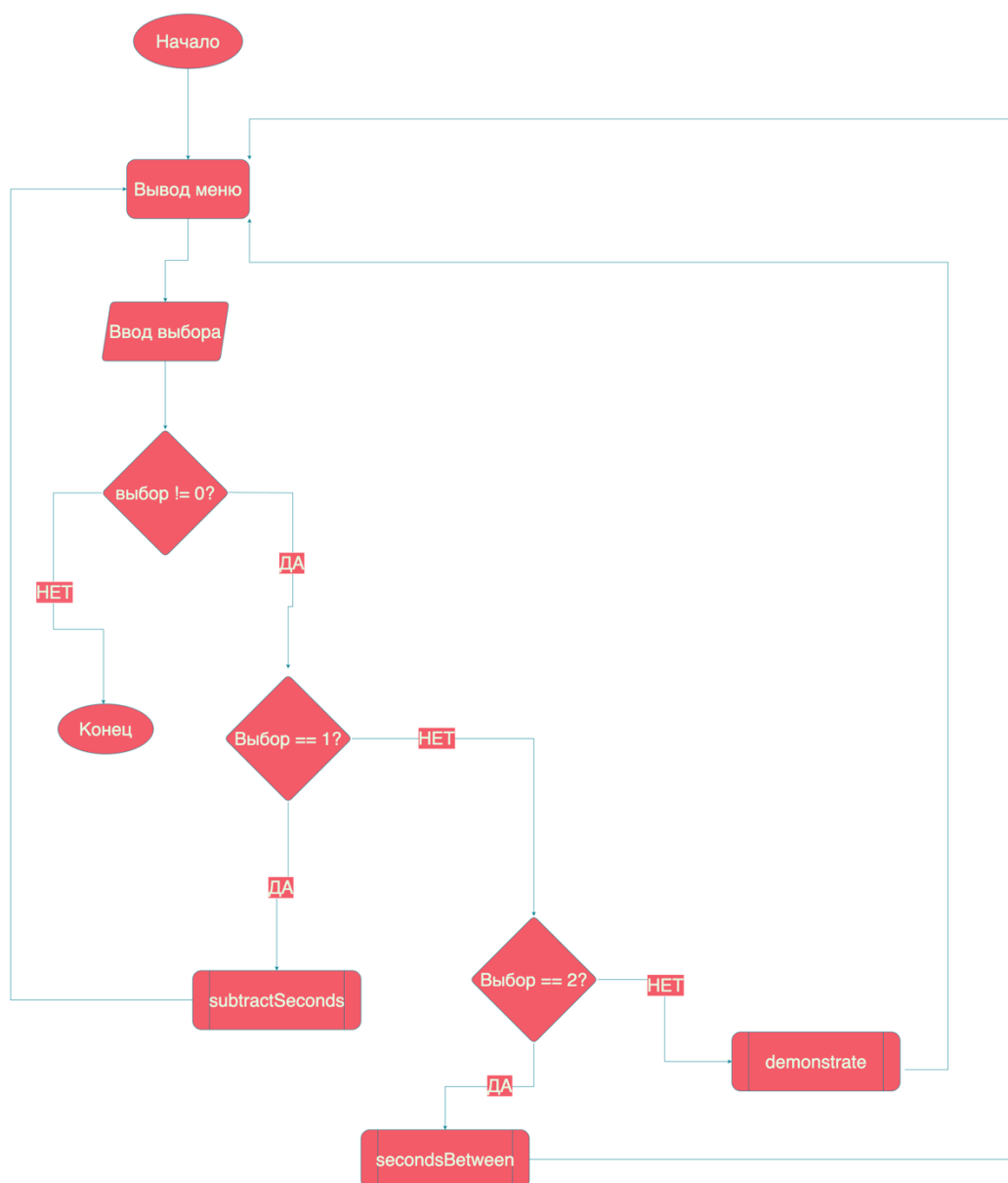


Рисунок 1 — Блок-схема для программы

Подпрограмма Timer принимает от пользователя значение времени в виде трёх отдельных чисел: часы (0–23), минуты (0–59) и секунды (0–59). Затем принимается количество секунд для вычитания. Введённое время переводится в секунды, выполняется вычитание, результат переводится обратно в формат ЧЧ:ММ:СС и выводится на экран. На рисунке 2 изображена блок-схема подпрограммы Timer.

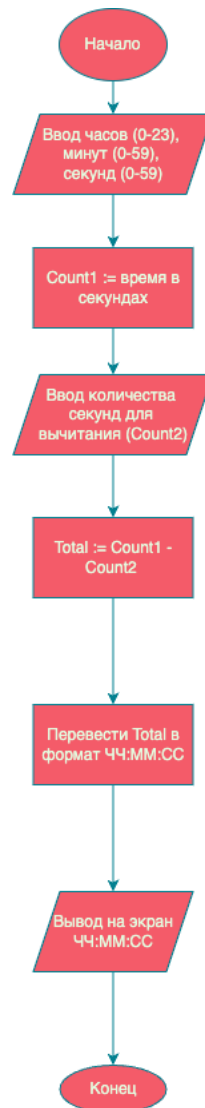


Рисунок 2 — Блок-схема для подпрограммы Timer

На рисунке 3 изображена блок-схема подпрограммы DeductSeconds. Подпрограмма принимает два момента времени, каждый из которых задаётся тремя числами аналогично подпрограмме Timer. Оба момента переводятся в секунды, вычисляется абсолютная разница между ними и результат выводится на экран в секундах.

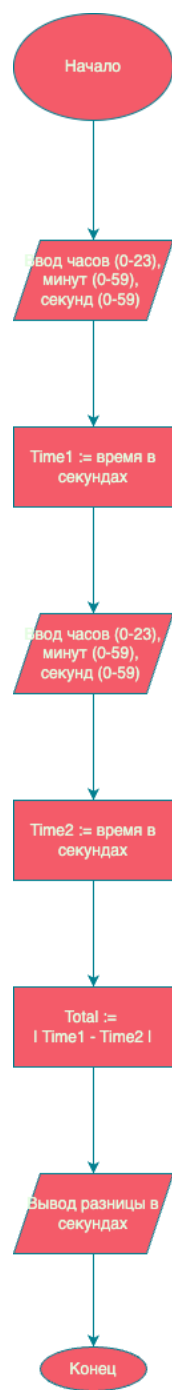


Рисунок 3 — Блок-схема подпрограммы DeductSeconds

2 ОПИСАНИЕ ПРОГРАММЫ

2.1 ОПИСАНИЕ СТРУКТУРЫ ВХОДНЫХ И ВЫХОДНЫХ ДАННЫХ

2.1.1 Входные данные

Программа принимает входные данные через консоль. Состав входных данных зависит от выбранного пункта меню.

Пункт 1 — Вычитание секунд из времени:

- Исходное время задаётся тремя отдельными целыми числами:
 - Часы: целое число от 0 до 23;
 - Минуты: целое число от 0 до 59;
 - Секунды: целое число от 0 до 59.
- Количество секунд для вычитания: неотрицательное целое число.

Пункт 2 — Подсчёт секунд между двумя моментами времени:

- Два момента времени; каждый задаётся тремя отдельными целыми числами в тех же диапазонах: часы (0–23), минуты (0–59), секунды (0–59)

Пункт 3 — Демонстрация всех операций:

Пользовательский ввод не требуется. Программа использует заранее заданные значения. Для всех числовых полей реализована проверка корректности ввода: при вводе значения вне допустимого диапазона или нечислового символа программа выводит сообщение об ошибке и повторно запрашивает ввод.

2.1.2 Выходные данные

Пункт 1 — Вычитание секунд из времени:

- Исходное время в формате ЧЧ:ММ:СС
- Количество вычитаемых секунд
- Результирующее время в формате ЧЧ:ММ:СС

Пункт 2 — Подсчёт секунд между двумя моментами времени:

- Первый момент времени в формате ЧЧ:ММ:СС
- Второй момент времени в формате ЧЧ:ММ:СС
- Разница между моментами в секундах
- Та же разница, представленная в виде часов, минут и секунд

Пункт 3 — Демонстрация всех операций:

- Вывод значений объектов Time, созданных разными конструкторами
- Результаты операций: вычитание секунд, разница между моментами, сравнение
- Содержимое объекта Group (массив объектов Time)
- Сообщения о вызовах конструкторов и деструкторов, демонстрирующие жизненный цикл объектов

2.2 ФУНКЦИОНАЛЬНАЯ СТРУКТУРА ПРОГРАММЫ

Программа состоит из трёх файлов реализации: `timer.cpp` (класс Time), `group.cpp` (класс Group), `main.cpp` (логика меню), и соответствующих заголовочных файлов `timer.h`, `group.h`. Класс Group является контейнером для объектов класса Time и управляет динамически выделенной памятью.

Класс Time (`timer.h`, `timer.cpp`) представляет момент времени в пределах суток. Содержит три закрытых поля: `hours`, `minutes`, `seconds`. В классе объявлены конструкторы, реализованы методы для работы со временем. Для работы с объектами используются перегруженные операторы.

Конструкторы:

- `Time()` — конструктор по умолчанию, инициализирует время значением 00:00:00;
- `Time(int h, int m, int s)` — конструктор с параметрами;
- `Time(int totalSeconds)` — конструктор из общего числа секунд;
- `Time(const Time& other)` — конструктор копирования.

Методы:

- setTime, setHours, setMinutes, setSeconds — установка значений полей;
- getHours, getMinutes, getSeconds — получение значений полей;
- toSeconds — переводит время в общее количество секунд;
- fromSeconds — восстанавливает время из общего числа секунд;
- deductSeconds — вычитает заданное количество секунд из времени;
- secondsBetween — возвращает абсолютную разницу в секундах между двумя моментами времени;
- toString — возвращает строковое представление в формате ЧЧ:ММ:СС;
- inputFromConsole — запрашивает ввод с клавиатуры с проверкой допустимых диапазонов значений;
- isValid — статический метод проверки корректности значений часов, минут и секунд;
- normalize — закрытый вспомогательный метод, приводит значения полей к допустимому диапазону после любой арифметической операции, обеспечивая корректность представления времени (например, 61 секунда преобразуется в 1 минуту 1 секунду).

Для работы с объектами используются перегруженные операторы:

- - — вычитает заданное количество секунд, возвращает новый объект Time;
- -= — вычитает секунды из текущего объекта;
- = — присваивание;
- == — проверка равенства двух моментов времени;
- < — проверка, что левый момент времени раньше правого;
- > — проверка, что левый момент времени позже правого;
- << — вывод времени в поток в формате ЧЧ:ММ:СС;

- >> — ввод времени из потока в формате ЧЧ:ММ:СС.

Класс Group (group.h, group.cpp) представляет динамический массив объектов Time. Содержит два закрытых поля: timers (указатель на массив) и size (размер массива). В классе объявлены конструкторы, реализованы методы для работы с массивом. Для работы с объектами используются перегруженные операторы: [] (доступ к элементу), = (присваивание с глубоким копированием).

Конструкторы:

- Group(int size) — создаёт массив объектов Time заданного размера;
- Group(const Group& other) — конструктор копирования с глубоким копированием.

Конструктор копирования и оператор = реализуют глубокое копирование, поскольку класс управляет динамической памятью (new/delete[]). Без них копирование объекта Group привело бы к двойному освобождению памяти.

Методы:

- setTimer — устанавливает объект Time по индексу;
- subtractSeconds — вычитает секунды из объекта по индексу;
- getSize — возвращает размер массива.

Модуль main.cpp реализует консольное меню и логику взаимодействия с пользователем. Для управления выводом диагностических сообщений конструкторов и деструктора используется глобальный флаг g_verbose. В режиме демонстрации (пункт 3 меню) флаг включается, что позволяет наблюдать жизненный цикл объектов Time.

Функции модуля:

- showMenu — выводит меню на экран;
- subtractSecondsOperation — запрашивает у пользователя момент времени и количество секунд, выполняет вычитание и выводит результат;

- `calculateSecondsBetween` — запрашивает два момента времени, вычисляет и выводит разницу в секундах и в формате часов, минут и секунд;
- `demonstrateAllOperations` — последовательно демонстрирует все конструкторы, перегруженные операторы и класс `Group`;
- `main` — точка входа, содержит цикл `do-while` для повторного отображения меню до выбора пункта «Выход».

2.3 ТЕСТИРОВАНИЕ ПРОГРАММЫ

Для подтверждения корректности работы программы было проведено тестирование всех реализованных операций. Проверялись как стандартные сценарии с корректными входными данными, так и граничные случаи: некорректный ввод, нулевые значения, переход через границу суток. Для каждого теста зафиксированы входные данные, ожидаемый и фактический результаты. Результаты тестирования и соответствующие снимки экрана представлены ниже.

1 Тестирование операции вычитания секунд (пункт 1 меню)

Тест 1.1 — корректный ввод, результат в пределах суток: Проверяется корректность вычитания секунд при стандартном вводе. Результат выполнения теста представлен на рисунке 4. Данные теста приведены в таблице 1.

Таблица 1 — Тестирование операции вычитания секунд. Корректный ввод

№	Входные данные	Ожидаемый результат	Фактический результат
1	Время: 04:32:47, секунды: 145	Оставшееся время: 04:30:22	Совпадает

```
1. Вычитание секунд из времени
2. Подсчёт секунд между двумя моментами
3. Демонстрация всех операций
0. Выход
=====
Выберите пункт меню: 1

--- Вычитание секунд из времени ---

Введите исходное время:
Введите часы (0-23): 04
Введите минуты (0-59): 32
Введите секунды (0-59): 47

Введённое время: 04:32:47

Введите количество секунд для вычитания: 145

=====
Результат:
Исходное время: 04:32:47
Вычитаемые секунды: 145
Оставшееся время: 04:30:22
=====
```

Рисунок 4 — Тестирование операции вычитания секунд. Корректный вывод

Тест 1.2 — вычитание с переносом через сутки. Код не ограничивает количество вычитаемых секунд. При вычитании большего количества секунд, чем представлено во введённом времени, метод `normalize()` автоматически выполняет перенос через границу суток. Результат выполнения теста представлен на рисунке 5. Данные теста приведены в таблице 2.

Таблица 2 — Тестирование операции вычитания секунд. Перенос через сутки

№	Входные данные	Ожидаемый результат	Фактический результат
1	Время: 00:05:00, секунды: 600	Оставшееся время: 23:55:00	Совпадает

```
--- Вычитание секунд из времени ---

Введите исходное время:
Введите часы (0-23): 0
Введите минуты (0-59): 5
Введите секунды (0-59): 0

Введённое время: 00:05:00

Введите количество секунд для вычитания: 600

=====
Результат:
Исходное время: 00:05:00
Вычитаемые секунды: 600
Оставшееся время: 23:55:00
=====
```

Рисунок 5 — Вычитание с переносом через сутки

Тест 1.3 — некорректный ввод времени. Проверяется корректность обработки некорректных входных данных. При вводе значения вне допустимого диапазона программа выводит сообщение об ошибке и повторно запрашивает ввод. Результат выполнения теста представлен на рисунке 6. Данные теста приведены в таблице 3.

Таблица 3 — Тестирование операции вычитания. Некорректный ввод

№	Входные данные	Ожидаемый результат	Фактический результат
1	Часы: 25	Сообщение об ошибке, повторный запрос	Совпадает
2	Минуты: 60	Сообщение об ошибке, повторный запрос	Совпадает

№	Входные данные	Ожидаемый результат	Фактический результат
3	Секунды: -1	Сообщение об ошибке, повторный запрос	Совпадает

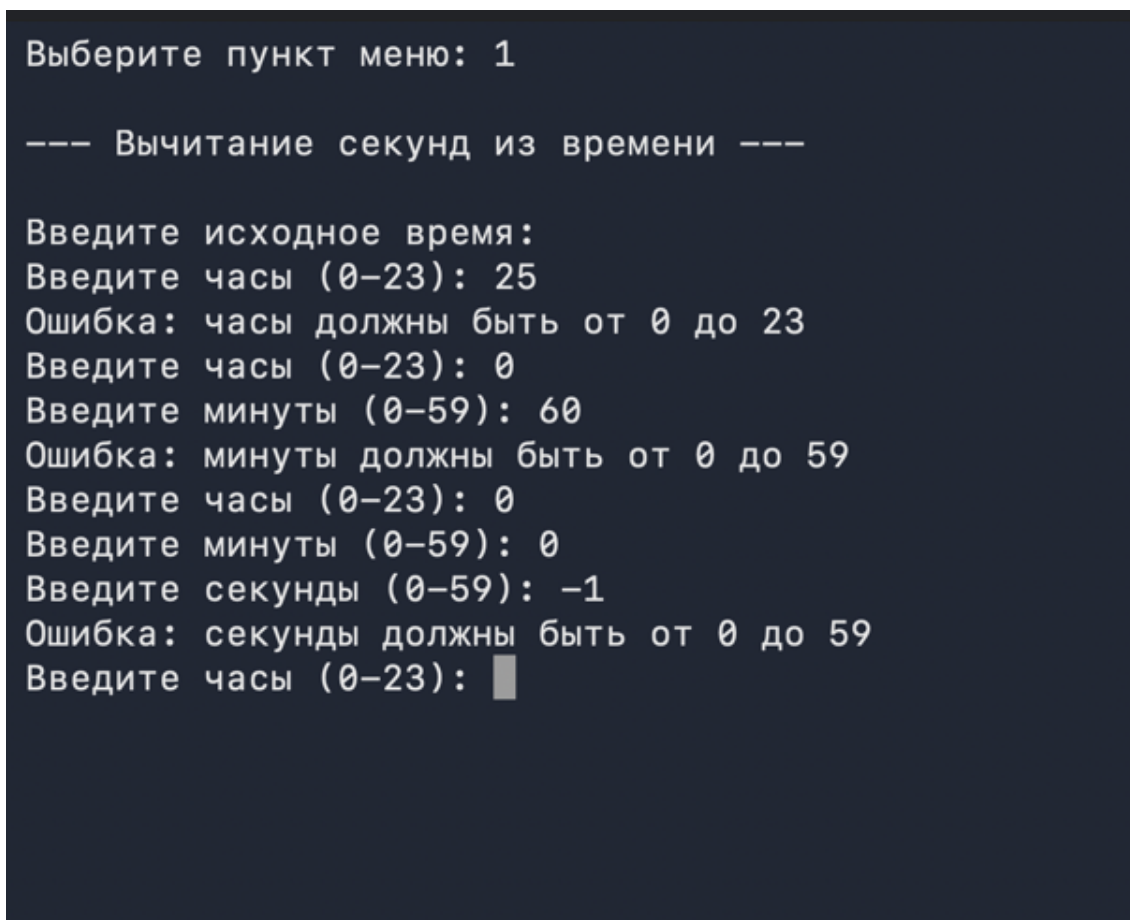


Рисунок 6 — Некорректный ввод времени

2 Тестирование операции подсчёта секунд между моментами (пункт 2 меню)

Тест 2.1 — корректный ввод. Проверяется корректность подсчёта секунд между двумя моментами времени при стандартном вводе. Результат выполнения теста представлен на рисунке 7. Данные теста приведены в таблице 4.

Таблица 4 — Тестирование операции подсчёта секунд. Корректный ввод

№	Входные данные	Ожидаемый результат	Фактический результат
1	Время 1: 12:10:12, Время 2: 18:45:00	Разница: 23688 сек. (6 ч. 34 мин. 48 сек.)	Совпадает

```

--- Подсчёт секунд между двумя моментами ---

Введите первый момент времени:
Введите часы (0-23): 12
Введите минуты (0-59): 10
Введите секунды (0-59): 12

Введите второй момент времени:
Введите часы (0-23): 18
Введите минуты (0-59): 45
Введите секунды (0-59): 0

=====
Результат:
Первый момент времени: 12:10:12
Второй момент времени: 18:45:00
Разница в секундах: 23688
Что составляет: 6 ч. 34 мин. 48 сек.
=====

```

Рисунок 7 — Тестирование операции подсчёта секунд между моментами. Корректный ввод

Тест 2.2 — моменты в обратном порядке. Метод `secondsBetween` использует `abs()`, поэтому порядок ввода не важен. Результат выполнения теста представлен на рисунке 8. Данные теста приведены в таблице 5.

Таблица 5 — Тестирование операции подсчёта секунд. Моменты в обратном порядке

№	Входные данные	Ожидаемый результат	Фактический результат
1	Время 1: 18:45:00, Время 2: 12:10:12	Разница: 23688 сек. (6 ч. 34 мин. 48 сек.)	Совпадает

```

--- Подсчёт секунд между двумя моментами ---

Введите первый момент времени:
Введите часы (0-23): 18
Введите минуты (0-59): 45
Введите секунды (0-59): 0

Введите второй момент времени:
Введите часы (0-23): 12
Введите минуты (0-59): 10
Введите секунды (0-59): 0

=====
Результат:
Первый момент времени: 18:45:00
Второй момент времени: 12:10:00
Разница в секундах: 23700
Что составляет: 6 ч. 35 мин. 0 сек.
=====

```

Рисунок 8 — Тестирование операции подсчёта секунд. Моменты в обратном порядке

Тест 2.3 — одинаковые моменты времени. Проверяется поведение программы при вводе двух одинаковых моментов времени. Разница между ними должна быть равна нулю. Результат выполнения теста представлен на рисунке 9. Данные теста приведены в таблице 6.

Таблица 6 — Тестирование операции подсчёта секунд. Одинаковые моменты времени

№	Входные данные	Ожидаемый результат	Фактический результат

1	Время 1: 10:00:00, Время 2: 10:00:00	Разница: 0 сек	Совпадает
---	---	----------------	-----------

```

--- Подсчёт секунд между двумя моментами ---

Введите первый момент времени:
Введите часы (0-23): 10
Введите минуты (0-59): 0
Введите секунды (0-59): 0

Введите второй момент времени:
Введите часы (0-23): 10
Введите минуты (0-59): 0
Введите секунды (0-59): 0

=====
Результат:
Первый момент времени: 10:00:00
Второй момент времени: 10:00:00
Разница в секундах: 0
Что составляет: 0 ч. 0 мин. 0 сек.
=====

```

Рисунок 9 — Тестирование операции подсчёта секунд. Одинаковые моменты времени

3 Тестирование меню

Тест 3.1 — некорректный пункт меню. Проверяется обработка некорректного ввода при выборе пункта меню: числа вне допустимого диапазона и нечисловые символы. Данные теста приведены в таблице 7. Результат выполнения теста представлен на рисунке 10.

Таблица 7 — Тестирование меню. Некорректный пункт меню

№	Входные данные	Ожидаемый результат	Фактический результат
---	----------------	---------------------	-----------------------

1	Выбор: 5	«Неверный выбор! Попробуйте снова.»	Совпадает
2	Выбор: -1	«Неверный выбор! Попробуйте снова.»	Совпадает
3	Выбор: abc	«Ошибка ввода! Попробуйте снова.»	Совпадает

```

=====
ПРОГРАММА РАБОТЫ СО ВРЕМЕНЕМ
=====
1. Вычитание секунд из времени
2. Подсчёт секунд между двумя моментами
3. Демонстрация всех операций
0. Выход
=====
Выберите пункт меню: 5

Неверный выбор! Попробуйте снова.

=====
ПРОГРАММА РАБОТЫ СО ВРЕМЕНЕМ
=====
1. Вычитание секунд из времени
2. Подсчёт секунд между двумя моментами
3. Демонстрация всех операций
0. Выход
=====
Выберите пункт меню: -1

Неверный выбор! Попробуйте снова.

=====
ПРОГРАММА РАБОТЫ СО ВРЕМЕНЕМ
=====
1. Вычитание секунд из времени
2. Подсчёт секунд между двумя моментами
3. Демонстрация всех операций
0. Выход
=====
Выберите пункт меню: abc
Ошибка ввода! Попробуйте снова.

```

Рисунок 10 — Тестирование меню. Некорректный пункт меню

Тест 3.2 — выход из программы. Проверяется корректность завершения программы при выборе пункта 0. Данные теста приведены в таблице 8. Результат выполнения теста представлен на рисунке 11.

Таблица 8 — Тестирование меню. Выход из программы

№	Входные данные	Ожидаемый результат	Фактический результат
1	Выбор: 0	«Завершение программы...»	Совпадает

```

=====
      ПРОГРАММА РАБОТЫ СО ВРЕМЕНЕМ
=====
1. Вычитание секунд из времени
2. Подсчёт секунд между двумя моментами
3. Демонстрация всех операций
0. Выход
=====
Выберите пункт меню: 0

Завершение программы...
Нажмите Enter, чтобы выйти...

```

Рисунок 11 — Тестирование меню. Выход из программы

4 Тестирование класса Group (пункт 3 меню — демонстрация)

Тест 4.1 — сохранение и доступ к элементам. Создаётся объект Group(2), в него записываются два объекта Time. Обращение через operator[] должно возвращать корректные значения. Данные теста приведены в таблице 9. Результат выполнения теста представлен на рисунке 12.

Таблица 9 — Тестирование класса Group. Сохранение и доступ к элементам

№	Входные данные	Ожидаемый результат	Фактический результат
1	setTimer(0, Time(10,30,0)), group[0]	10:30:00	Совпадает
2	setTimer(1, Time(14,45,30)), group[1]	14:45:30	Совпадает

```
7. Демонстрация класса Group:
-----
Вызван конструктор Group с размером 2
Вызван конструктор по умолчанию
Вызван конструктор по умолчанию
Вызван конструктор с параметрами (10, 30, 0)
Вызван конструктор с параметрами (14, 45, 30)
group[0]: 10:30:00
group[1]: 14:45:30

--- Конец демонстрации ---
(Обратите внимание на вызовы деструкторов)
Вызван деструктор Group для массива из 2 элементов

=====
ПРОГРАММА РАБОТЫ СО ВРЕМЕНЕМ
=====
1. Вычитание секунд из времени
2. Подсчёт секунд между двумя моментами
3. Демонстрация всех операций
0. Выход
=====
Выберите пункт меню: █
```

Рисунок 12 — Тестирование класса Group. Сохранение и доступ к элементам

Тест 4.2 — конструктор копирования и оператор присваивания. При копировании объекта Group изменение копии не должно затрагивать оригинал (глубокое копирование). Данные теста приведены в таблице 10.

Таблица 10 — Тестирование класса Group. Конструктор копирования и оператор присваивания

№	Входные данные	Ожидаемый результат	Фактический результат
1	Копирование Group, изменение копии	Оригинал не изменяется	Совпадает

5 Тестирование метода toString и оператора <<

Проверяется корректность форматирования вывода: метод toString и оператор << должны возвращать время в формате ЧЧ:ММ:СС с ведущими нулями. Данные теста приведены в таблице 11. Результат выполнения теста представлен на рисунке 13.

Таблица 11 — Тестирование метода toString и оператора <<

№	Входные данные	Ожидаемый результат	Фактический результат
1	Time(4, 5, 9)	04:05:09 (ведущие нули)	Совпадает
2	Time(0, 0, 0)	00:00:00	Совпадает
3	Time(23, 59, 59)	23:59:59	Совпадает

```
8. Тестирование toString и оператора <<:
-----
Вызван конструктор с параметрами (4, 5, 9)
04:05:09
Вызван деструктор для времени 04:05:09
Вызван конструктор с параметрами (0, 0, 0)
00:00:00
Вызван деструктор для времени 00:00:00
Вызван конструктор с параметрами (23, 59, 59)
23:59:59
Вызван деструктор для времени 23:59:59
```

Рисунок 13 — Тестирование метода toString и оператора <<

2.4 РУКОВОДСТВО ПОЛЬЗОВАТЕЛЯ

После запуска исполняемого файла на экране отображается главное меню. Пример вывода меню представлен на рисунке 14.

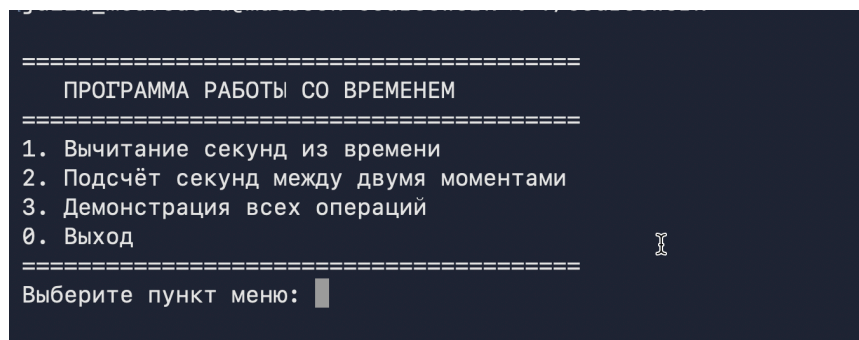


Рисунок 14 — Первый запуск программы

Программа работает в цикле: после выполнения любой операции меню отображается повторно. Для завершения работы необходимо выбрать пункт 0.

Пункт 1 — Вычитание секунд из времени

1. Введите исходное время тремя отдельными числами:
 - a. Часы: целое число от 0 до 23;
 - b. Минуты: целое число от 0 до 59;
 - c. Секунды: целое число от 0 до 59.
2. Введите количество секунд для вычитания (неотрицательное целое число). При вводе значения вне допустимого диапазона или нечислового символа программа выводит сообщение об ошибке и повторно запрашивает ввод.

Пример вывода результата отображен на рисунке 15.


```
ПРОГРАММА РАБОТЫ СО ВРЕМЕНЕМ
=====
1. Вычитание секунд из времени
2. Подсчёт секунд между двумя моментами
3. Демонстрация всех операций
0. Выход
=====
Выберите пункт меню: 1

--- Вычитание секунд из времени ---

Введите исходное время:
Введите часы (0-23): 4
Введите минуты (0-59): 32
Введите секунды (0-59): 47

Введённое время: 04:32:47

Введите количество секунд для вычитания: 145

=====
Результат:
Исходное время: 04:32:47
Вычитаемые секунды: 145
Оставшееся время: 04:30:22
=====
```

Рисунок 15 — Вычитание секунд из времени

Пункт 2 — Подсчёт секунд между двумя моментами

1. Введите первый момент времени тремя отдельными числами (часы, минуты, секунды).
2. Введите второй момент времени тремя отдельными числами (часы, минуты, секунды).

Порядок ввода моментов не важен: программа возвращает абсолютную разницу. Пример вывода результата представлен на рисунке 16.


```

--- Подсчёт секунд между двумя моментами ---

Введите первый момент времени:
Введите часы (0–23): 12
Введите минуты (0–59): 10
Введите секунды (0–59): 12

Введите второй момент времени:
Введите часы (0–23): 18
Введите минуты (0–59): 45
Введите секунды (0–59): 00

=====
Результат:
Первый момент времени: 12:10:12
Второй момент времени: 18:45:00
Разница в секундах: 23688
Что составляет: 6 ч. 34 мин. 48 сек.
=====

```

Рисунок 16 — Подсчёт секунд между двумя моментами

Пункт 3 — Демонстрация всех операций

Пользовательский ввод не требуется. Программа автоматически демонстрирует:

1. работу всех конструкторов класса Time;
2. преобразование времени в секунды;
3. вычитание секунд и оператор -=;
4. подсчёт разницы между двумя моментами;
5. операторы сравнения (<, >, ==);
6. класс Group (массив объектов Time);
7. форматирование вывода: ведущие нули в toString и операторе <<.

В этом режиме также отображаются сообщения о вызовах конструкторов и деструкторов, что позволяет наблюдать жизненный цикл объектов.

Пункт 0 — Выход

Программа выводит сообщение «Завершение программы...», затем предлагает нажать Enter для закрытия окна консоли.

3 ПАРАМЕТРЫ ЧЛЕНОВ-ФУНКЦИЙ. СПОСОБЫ ПЕРЕДАЧИ ЗНАЧЕНИЙ В ТЕЛО И ИЗ ТЕЛА ФУНКЦИИ

В языке программирования C++ функции играют важную роль в структуре программы, позволяя организовать выполнение повторяющихся операций в удобной и понятной форме. Основной задачей функции является получение входных данных, выполнение некоторых вычислений или операций и возврат результата. Одним из важнейших аспектов при работе с функциями является понимание параметров, передаваемых в функции, а также методов их передачи в тело функции и получения результатов. Рассмотрим основные аспекты этой темы, включая способы передачи параметров, их особенности и влияние на производительность программы.

3.1 ПАРАМЕТРЫ ЧЛЕНОВ-ФУНКЦИЙ В C++

Параметры функции представляют собой переменные, которые используются для передачи данных в функцию или из неё. В языке C++ параметры делятся на два типа:

1. Параметры функции — это переменные, которые передаются в функцию при её вызове.
2. Члены функции — это члены класса, к которым доступ можно получить через вызовы методов данного класса.

Методы-члены классов могут иметь параметры, как и обычные функции, однако существует особенность их передачи: эти параметры часто являются полями класса или объектами этого класса. Кроме того, методы-члены могут быть как константными, так и неконстантными, что накладывает ограничения на их использование в контексте изменения состояний объектов.

Рассмотрим пример:

```
#include <iostream>

class MyClass {
public:
```

```

int x;
MyClass(int val) : x(val) {}
void printX() const { // Константный метод
    std::cout << "x = " << x << std::endl;
}
void setX(int val) { // Неконстантный метод
    x = val;
}
};

int main() {
    MyClass obj(5);
    obj.printX(); // x = 5
    obj.setX(10);
    obj.printX(); // x = 10
    return 0;
}

```

В этом примере метод `setX` изменяет значение члена класса `x`, а метод `printX` просто выводит его на экран, не изменяя состояния объекта. Параметры `setX` и `printX` — это значения, передаваемые в функции.

Неявный параметр `this`

В C++ каждый нестатический метод класса имеет скрытый параметр — указатель `this`, который автоматически передаётся при вызове и указывает на объект, для которого вызван метод. Он не указывается явно в сигнатуре, но доступен внутри любого нестатического метода.

Рассмотрим пример использования `this`:

```

#include <iostream>

class Time {
private:
    int hours;
    int minutes;

```

public:

```
    Time(int h, int m) : hours(h), minutes(m) {}  
    // this явно используется для разрешения конфликта имён  
    void setHours(int hours) {  
        this->hours = hours; // this->hours — поле, hours — параметр  
    }  
    // Возврат *this позволяет строить цепочки вызовов  
    Time& setMinutes(int minutes) {  
        this->minutes = minutes;  
        return *this;  
    }  
    void print() const {  
        std::cout << hours << ":" << minutes << std::endl;  
    }  
};  
int main() {  
    Time t(10, 30);  
    t.setHours(12).setMinutes(45); // Цепочка вызовов благодаря *this  
    t.print(); // 12:45  
    return 0;  
}
```

В примере `this->hours` используется для разграничения поля класса и параметра с одинаковым именем. Возврат `*this` по ссылке позволяет строить цепочки вызовов методов. В константных методах (`const`) указатель `this` имеет тип `const ClassName*`, поэтому такой метод не может изменять поля объекта.

3.2 СПОСОБЫ ПЕРЕДАЧИ ЗНАЧЕНИЙ В ТЕЛО ФУНКЦИИ

В языке C++ существуют различные способы передачи значений в тело функции. Рассмотрим их более подробно.

1. Передача по значению:

Это самый простой способ передачи параметров в функцию. Значение аргумента копируется в параметр функции. Любые изменения параметра внутри функции не затрагивают оригинальную переменную в вызывающем коде.

Пример:

```
void changeValue(int a) {  
    a = 10;  
}  
  
int main() {  
    int x = 5;  
    changeValue(x);  
    std::cout << "x = " << x << std::endl; // x = 5  
    return 0;  
}
```

В этом примере значение переменной `x` передается в функцию `changeValue` по значению. Функция изменяет копию переменной, но оригинальная переменная в `main()` остается неизменной.

2. Передача по ссылке:

При передаче параметра по ссылке функция получает доступ к оригинальной переменной, и любые изменения будут отражаться на вызывающем коде. Это экономит память, так как не создается копия параметра, особенно если передаются большие объекты.

Пример:

```
void changeValue(int& a) {  
    a = 10;  
}  
  
int main() {  
    int x = 5;
```

```

changeValue(x);
std::cout << "x = " << x << std::endl; // x = 10
return 0;
}

```

В этом случае переменная `x` в функции `main()` изменяется, так как параметр передается по ссылке.

3. Передача по указателю:

Подобно передаче по ссылке, при передаче параметра по указателю функция получает доступ к оригинальной переменной, но в отличие от ссылки, указатели могут быть `nullptr`, что требует дополнительной проверки.

Пример:

```

void changeValue(int* a) {
    if (a != nullptr) {
        *a = 10;
    }
}

int main() {
    int x = 5;
    changeValue(&x);
    std::cout << "x = " << x << std::endl; // x = 10
    return 0;
}

```

В этом примере указатель передается в функцию, и с помощью оператора разыменования изменяется значение переменной.

4. Передача по константной ссылке (`const &`):

В отличие от передачи по значению, объект не копируется, что важно для производительности при работе со сложными объектами. Ключевое слово `const` гарантирует, что функция не изменит переданный объект. Это наиболее рекомендуемый способ передачи объектов классов в методы.

Пример:

```

#include <iostream>

class Time {
public:
    int hours, minutes, seconds;

    Time(int h, int m, int s) : hours(h), minutes(m), seconds(s) {}
};

// Принимает объекты по const& — не копирует, не изменяет
int secondsBetween(const Time& t1, const Time& t2) {
    int s1 = t1.hours * 3600 + t1.minutes * 60 + t1.seconds;
    int s2 = t2.hours * 3600 + t2.minutes * 60 + t2.seconds;
    return (s1 > s2) ? (s1 - s2) : (s2 - s1);
}

int main() {
    Time morning(8, 0, 0);
    Time evening(18, 30, 0);
    std::cout << "Разница: " << secondsBetween(morning, evening) << " сек."
<< std::endl;

    return 0;
}

```

Использование `const&` вместо передачи по значению особенно важно в методах класса: позволяет вызывать метод для константных объектов и не создаёт лишних копий. Именно так реализован метод `secondsBetween` в классе `Time` данной курсовой работы.

3.3 СПОСОБЫ ПЕРЕДАЧИ ЗНАЧЕНИЙ ИЗ ТЕЛА ФУНКЦИИ

Для возврата значений из тела функции в C++ используются разные методы:

1. Возврат значения:

Это стандартный способ возврата результата из функции. Функция возвращает значение через `return`, и это значение может быть присвоено переменной в вызывающем коде.

Пример:

```
int add(int a, int b) {  
    return a + b;  
}  
  
int main() {  
    int result = add(5, 3);  
    std::cout << "Result = " << result << std::endl; // Result = 8  
    return 0;  
}
```

2. Возврат через ссылку:

Функция может возвращать ссылку на существующую переменную. Однако, если функция возвращает локальную переменную по ссылке, это приведет к неопределенному поведению, так как локальная переменная будет уничтожена после выхода из функции.

Пример:

```
int& getMax(int& a, int& b) {  
    return (a > b) ? a : b;  
}  
  
int main() {  
    int x = 5, y = 3;  
    getMax(x, y) = 10; // Меняет значение x  
    std::cout << "x = " << x << std::endl; // x = 10  
    return 0;  
}
```

Именно так реализованы операторы `=` и `-=` в классе `Time` данной курсовой работы — они возвращают `Time&`, что позволяет использовать их в цепочках присваивания.

3. Возврат через указатель:

Функция может возвращать указатель на объект, что позволяет модифицировать объект в вызывающем коде. Также важно убедиться, что возвращаемый указатель указывает на действительную область памяти.

Пример:

```
int* getPointer() {  
    static int value = 5;  
    return &value;  
}  
  
int main() {  
    int* ptr = getPointer();  
    std::cout << "Value = " << *ptr << std::endl; // Value = 5  
    return 0;  
}
```

Здесь используется ключевое слово `static` для сохранения значения переменной между вызовами функции.

Заключение

Таким образом, в ходе выполнения курсовой работы была создана программа для управления временными интервалами, которая позволяет корректно обрабатывать операции вычитания секунд и подсчёта временных промежутков. Применение объектно-ориентированного подхода продемонстрировало преимущества использования классов и методов для структурирования кода и упрощения работы с временными данными.

При этом особое внимание уделялось анализу и практическому использованию способов передачи параметров. Умение выбирать подходящий способ передачи данных — по значению, по ссылке, по указателю или по константной ссылке — не только позволяет оптимизировать производительность программы, но и делает код более понятным и управляемым. Это знание важно для разработки эффективного и надёжного программного обеспечения, что особенно актуально при работе с ресурсозатратными операциями или большими объёмами данных.

В результате выполненной работы были закреплены как практические навыки объектно-ориентированного программирования, так и теоретические знания о параметрах функций, что создаёт основу для более сложных проектов и глубинного понимания работы функций в C++.

Список литературы

1. Дейтел П.Дж. Как программировать на C++: Пятое издание. Пер. с англ. М.: ООО «Бином-Пресс», 2008.
2. Кнут Д.Р. Искусство программирования. Т. 1. Алгоритмы. 3-е изд. М.: Вильямс, 2018.
3. Липпман С., Лажой Ж., Муллиган Б.Э. Язык программирования C++. Базовый курс. 5-е изд. М.: Диалектика-Вильямс, 2018.
4. Лоспинозо Дж. C++ для профи. М.: Питер, 2021.
5. Страуструп Б. C++ Programming Language (или Язык программирования C++). 4-е изд. М.: Бином, 2022.